Abdulmohsen Almunayes

RED ID: 823343580

2/28/2024

LAB 4 REPORT

What does your program do?

The program gradually illuminates an LED when a button on the Atmel board is pressed, and gradually dims the LED when the button is released.

What variables/registers does your program use?

The program utilizes registers TCCR0A/B, OCR0A/B, TIFR, DDRB, OCF0A/B, PORTB, and PINB.

What is the main algorithm/logic of your program?

My program initializes the LED pin (PB5) as an output and the button pin (PB7) as an input, with a pull-up resistor enabled for the button pin. In the main loop, it continuously monitors the button state to adjust the LED brightness. A static variable brightness tracks the current brightness level. When the button is pressed, the run_timer() function is invoked with the current brightness value, incrementing it if below 100. Conversely, when the button is released, the function is called with the current brightness, decrementing it if above 0. Within the run_timer() function, Timer/Counter0 is configured for Clear Timer on Compare (CTC) mode, defining the timer period via OCR0A and the LED brightness via OCR0B. The prescale is set to 1024, and the LED is turned on. The program waits for the timer to reach the desired brightness level before clearing the output compare flag and turning off the LED. Finally, it waits for the timer cycle to complete before clearing the output compare flag.

What external hardware connections did you make?

The connections are established between the PC and the Atmel board.

C CODE:

```
#include <avr/io.h>
#include <avr/interrupt.h>

#define F_CPU 16000000UL  // AVR clock frequency
#define LED_PIN PB5
#define BUTTON_PIN PB7

void setup_LED_Button() {
DDRB |= (1 << LED_PIN);   // Set LED pin as output
DDRB &= ~(1 << BUTTON_PIN);  // Set push button pin as input
PORTB |= (1 << BUTTON_PIN);  // Enable pull-up resistor for button pin
}
void run_timer(int value);

int main(void) {

setup_LED_Button();

while (1) {
static int brightness = 0;

if (!(PINB & (1 << BUTTON_PIN))) {
run_timer(brightness);
if (brightness < 100) {
brightness++;
}
} else {
run_timer(brightness);
if (brightness > 0) {
brightness--;
}
}
}
}


void run_timer(int value) {
TCCR0A |= (1 << WGM01); // Set Timer0 for CTC mode
OCR0A = 0x9B;           // Set output compare register A value
OCR0B = value;          // Set output compare register B value
TCCR0B |= (1 << CS00) | (1 << CS02); // Set prescale to 1024
PORTB |= (1 << LED_PIN);   // Turn on LED
```

```
while (!(TIFR0 & (1 << OCF0B))); // Wait until output compare match B
TIFR0 |= (1 << OCF0B);  // Clear output compare flag B
PORTB &= ~(1 << LED_PIN);    // Turn off LED

while (!(TIFR0 & (1 << OCF0A))); // Wait until output compare match A
TIFR0 |= (1 << OCF0A);  // Clear output compare flag A
}
```